

**ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT
MÁY TÍNH**



Báo cáo Bài Tập Lớn

Game sắp xếp block

GVHD: TS. Trương Vĩnh Lân

SVTH 1: Phạm Ngọc Long	2211894
SVTH 2: Nguyễn Ngọc Duy	2210522
SVTH 3: Võ Quốc Phong	2352913
SVTH 4: Nguyễn Tấn Đạt	2352234

TP. HỒ CHÍ MINH, 2026

Mục lục

1	Giới thiệu	2
1.1	Bối cảnh bài toán	2
1.2	Phát biểu bài toán	2
1.3	Thông tin tra cứu	3
2	Biểu diễn & tìm kiếm	4
2.1	Mô hình hóa bài toán	4
2.2	Thuật toán tìm kiếm sử dụng	7
2.2.1	Best-first Search cho một lượt	7
3	Thiết kế heuristic cho hệ thống tìm kiếm	10
3.1	Vai trò của heuristic trong hệ thống	10
3.2	Hàm heuristic được sử dụng	10
3.3	Đánh giá thực nghiệm các trọng số (Ablation Study)	12
3.3.1	Kịch bản Baseline	12
3.3.2	Kịch bản 1: Loại bỏ yếu tố kiểm soát phân mảnh($w_1 = 0$)	12
3.3.3	Kịch bản 2: Loại bỏ yếu tố kiểm soát diện tích bao phủ($w_2 = 0$)	12
3.3.4	Kịch bản 3: Tối đa hóa cục bộ việc xóa hàng ($w_3 \gg w_1, w_2$)	13
4	Biểu diễn và suy luận tri thức	14
4.1	Tri thức bổ sung trong hệ thống	14
4.2	Cơ chế tie-break trong hệ thống	15
5	Mạng Bayes và suy diễn xác suất rủi ro	16
5.1	Mục tiêu của chương	16
5.2	Cơ sở lý thuyết	16
5.3	Định nghĩa các biến ngẫu nhiên	17
5.4	Phạm vi thông tin	18
5.5	Cấu trúc mạng Bayes	18
5.6	Bảng xác suất có điều kiện CPD	19

5.7	Suy diễn xác suất	19
5.8	Kết quả thực nghiệm của module Bayes	20
5.9	Vai trò của module Bayes trong hệ thống	21
5.10	Độ phủ dữ liệu và hướng mở rộng	21
6	Học máy	23
6.1	Vai trò của học máy trong hệ thống	23
6.2	Sinh dữ liệu huấn luyện bằng Teacher Signal	23
6.2.1	Sinh trạng thái hợp lệ	24
6.2.2	Gán nhãn bằng BFS Oracle	24
6.2.3	Cấu hình sinh dữ liệu	24
6.3	Trích xuất đặc trưng	25
6.4	EDA - Phân tích khám phá dữ liệu	26
6.4.1	Phân phối nhãn và Quan hệ của delta_holes với nhãn	26
6.4.2	Ma trận đặc trưng	26
6.4.3	Boxplot: điểm số trung bình theo nhãn	27
6.5	Mô hình học máy	28
6.5.1	Decision Tree Classifier	28
6.5.2	Random Forest Classifier	28
6.6	Huấn luyện và đánh giá mô hình	29
6.6.1	Phân chia dữ liệu	29
6.6.2	Kết quả trên tập kiểm tra	29
6.6.3	Đánh giá bằng 5-fold Cross-Validation	30
6.6.4	Tầm quan trọng của đặc trưng	31
6.7	So sánh và lựa chọn mô hình	32
6.7.1	So sánh tổng quan	32
6.7.2	Quyết định chọn mô hình	32
6.7.3	Luật suy diễn của Decision Tree	34
6.8	Phân tích kết quả và giới hạn mô hình	35
6.8.1	Phân tích Precision – Recall	35
6.8.2	Nguyên nhân F1-score bị giới hạn ở mức ≈ 0.50	35
6.9	Tích hợp mô hình vào Best-first Search	36
6.9.1	Kết quả thực nghiệm so sánh BFS thuần vs BFS + DT	37
6.9.2	Kết quả mô phỏng 3 lượt chơi liên tiếp	38
6.10	Các file đầu ra của module học máy	38
6.11	Nhận xét tổng kết	39
7	Kết luận	40
7.1	Tóm tắt kết quả đạt được	40
7.2	Kết quả thực nghiệm	i
7.3	Hạn chế	i

7.4	Hướng phát triển	i
A	Phân chia công việc nhóm	ii
B	Lịch sử sửa đổi	iii
C	Lịch sử cuộc họp	iv

Chương 1

Giới thiệu

Trong bài tập lớn này, nhóm lựa chọn xây dựng một bài toán xếp khối trên bàn cờ dưới góc nhìn của trí tuệ nhân tạo. Bài toán có thể được mô hình hóa rõ ràng bằng các thành phần cơ bản như trạng thái, hành động và mục tiêu. Từ đó, hệ thống có thể áp dụng các phương pháp tìm kiếm để lựa chọn cách đặt khối phù hợp nhằm đạt được kết quả tốt nhất.

1.1 Bối cảnh bài toán

Bài toán xếp các khối hình học vào một không gian có giới hạn là một dạng bài toán quen thuộc trong trí tuệ nhân tạo và tối ưu hóa. Trong những bài toán này, mỗi quyết định đặt khối ở hiện tại đều ảnh hưởng trực tiếp đến khả năng sắp xếp các khối tiếp theo, vì vậy cần có một chiến lược hợp lý thay vì chỉ chọn ngẫu nhiên.

Trong đề tài này, nhóm xét bài toán trên một bàn cờ kích thước 8×8 . Quá trình chơi được chia thành nhiều lượt, và ở mỗi lượt hệ thống cung cấp 4 khối hình học để đặt lên bàn cờ sao cho hợp lệ. Chỉ khi đặt hết 4 khối của lượt hiện tại thì hệ thống mới cung cấp 4 khối tiếp theo. Sau mỗi lần đặt, nếu một hàng hoặc một cột được lấp đầy hoàn toàn thì hàng hoặc cột đó sẽ được xóa khỏi bàn cờ, đồng thời người chơi nhận được điểm số tương ứng. Vì vậy, bài toán không chỉ yêu cầu đặt khối đúng mà còn cần lựa chọn cách đặt để tối đa hóa điểm đạt được trong toàn bộ số lượt chơi đã cho.

1.2 Phát biểu bài toán

Bài toán được thực hiện trên một bàn cờ kích thước 8×8 , trong đó mỗi ô có thể ở trạng thái trống hoặc đã được chiếm. Trong quá trình chơi, hệ thống tiến hành qua một số lượt xác định trước; tại mỗi lượt, hệ thống cung cấp 4 khối hình học và cần

lựa chọn cách đặt các khối này lên bàn cờ sao cho hợp lệ.

Mỗi khối chỉ được đặt tại những vị trí phù hợp với kích thước bàn cờ và không được chồng lấp lên các ô đã có trước đó. Sau khi một khối được đặt, nếu một hàng hoặc một cột trên bàn cờ được lấp đầy hoàn toàn thì các ô trên hàng hoặc cột đó sẽ bị xóa khỏi bàn cờ. Điểm số đạt được được tính dựa trên số ô biến mất sau mỗi lần xóa hàng hoặc cột. Khi toàn bộ 4 khối của một lượt đã được đặt xong, hệ thống sẽ chuyển sang lượt tiếp theo và cung cấp 4 khối mới.

Nhiệm vụ của bài toán là tìm ra cách sắp xếp các khối trên bàn cờ sao cho vừa đảm bảo tính hợp lệ, vừa đạt được tổng điểm cao nhất sau toàn bộ số lượt chơi. Đây là cơ sở để xây dựng mô hình bài toán và áp dụng các thuật toán tìm kiếm trong các phần tiếp theo.

1.3 Thông tin tra cứu

- Link GG Colab: https://colab.research.google.com/drive/1-7p8F6Q2b_-ekDuZ0juHKjkqIsp6AZJb?usp=sharing
- Link Datasets: tự sinh

Chương 2

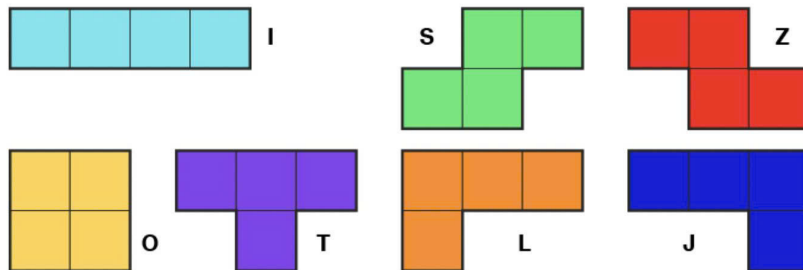
Biểu diễn & tìm kiếm

2.1 Mô hình hóa bài toán

Để đơn giản hóa, các block trong đề bài ban đầu chỉ giới hạn trong các khối Tetriminos dưới đây:

Tetriminos

Tetris shapes and their relative colors and letters



Hình 2.1: Tetriminos

Tuy nhiên, để trò chơi linh hoạt hơn, các block được phép xoay theo các góc cố định:

Listing 2.1: Tetriminos nhưng có phiên bản xoay

```
1 base = {  
2     'O': np.array([[1, 1],  
3                   [1, 1]]),  
4     'I': np.array([[1, 1, 1, 1]]),  
5     'S': np.array([[0, 1, 1],  
6                   [1, 1, 0]]),  
7     'Z': np.array([[1, 1, 0],
```

```

8         [0, 1, 1]]),
9     'L': np.array([[0, 0, 1],
10                    [1, 1, 1]]),
11     'J': np.array([[1, 0, 0],
12                    [1, 1, 1]]),
13     'T': np.array([[0, 1, 0],
14                    [1, 1, 1]])
15 }
16
17 # rotate base
18 TETRIMINOS = {
19     'O': base['O'],
20
21     'I_0': base['I'],
22     'I_90': np.rot90(base['I']),
23
24     'S_0': base['S'],
25     'S_90': np.rot90(base['S']),
26     'S_180': np.rot90(base['S'], 2),
27     'S_270': np.rot90(base['S'], 3),
28
29     'Z_0': base['Z'],
30     'Z_90': np.rot90(base['Z']),
31     'Z_180': np.rot90(base['Z'], 2),
32     'Z_270': np.rot90(base['Z'], 3),
33
34     'L_0': base['L'],
35     'L_90': np.rot90(base['L']),
36     'L_180': np.rot90(base['L'], 2),
37     'L_270': np.rot90(base['L'], 3),
38
39     'J_0': base['J'],
40     'J_90': np.rot90(base['J']),
41     'J_180': np.rot90(base['J'], 2),
42     'J_270': np.rot90(base['J'], 3),
43
44     'T_0': base['T'],
45     'T_90': np.rot90(base['T']),
46     'T_180': np.rot90(base['T'], 2),
47     'T_270': np.rot90(base['T'], 3),
48 }

```

Mặc dù trò chơi tổng thể diễn ra qua nhiều lượt, trong phạm vi đề tài này nhóm

chỉ xây dựng bài toán tìm kiếm trên **một lượt chơi**. Ở đầu mỗi lượt, hệ thống nhận trạng thái hiện tại của bàn cờ cùng với 4 khối cần đặt, sau đó sử dụng thuật toán tìm kiếm để xác định thứ tự đặt và vị trí đặt phù hợp cho các khối trong lượt đó.

Bài toán được mô hình hóa như sau:

1. State (Trạng thái) - S :

Một trạng thái $s \in S$ đại diện cho cấu hình của trò chơi tại một thời điểm trong một lượt, bao gồm:

- *Board*: Ma trận nhị phân kích thước 8×8 , trong đó $Board_{i,j} = 1$ nếu ô đã có khối và $Board_{i,j} = 0$ nếu ô trống.
- *Blocks*: Danh sách các khối còn lại cần đặt trong lượt hiện tại.

2. Action (Hành động) - $A(s)$:

Hành động $a \in A(s)$ là việc chọn **một khối bất kỳ** trong danh sách $s.Blocks$ và đặt vào một vị trí hợp lệ trên bàn cờ:

- $a = (b, x, y)$ với $b \in s.Blocks$ là khối được chọn, và (x, y) là tọa độ góc trên bên trái của khối trên *Board*.
- **Điều kiện hợp lệ**: khối không nằm ngoài biên bàn cờ 8×8 và không chồng lấp lên các ô đã có giá trị 1.

Như vậy, trong mỗi trạng thái, thuật toán có thể lựa chọn cả **loại khối** lẫn **vị trí đặt**, thay vì buộc phải đặt theo thứ tự cố định.

3. Goal Test (Kiểm tra mục tiêu):

Vì bài toán tìm kiếm chỉ được xét trên một lượt, trạng thái mục tiêu là khi toàn bộ các khối của lượt đó đã được đặt xong:

$$Goal(s) = \begin{cases} True & \text{nếu } s.Blocks = \emptyset \\ False & \text{nếu ngược lại} \end{cases} \quad (2.1)$$

Ngoài ra, một trạng thái cũng được xem là kết thúc sớm nếu không còn hành động hợp lệ nào để tiếp tục đặt khối.

4. Cost (Hàm chi phí trọng số) - $C(s, a, s')$:

Để tối ưu hóa cách chơi, chi phí được định nghĩa dưới dạng hàm phạt (*Penalty*). Một nước đi dẫn đến trạng thái s' càng “xấu” thì chi phí càng cao. Trong đề tài này, chi phí được xây dựng dựa trên ba đặc trưng chính: số

thành phần liên thông còn lại, diện tích bao phủ của các khối đã đặt, và số hàng/cột được xóa. Công thức chi phí được xác định như sau:

$$Cost(s, a, s') = w_1 \cdot \Delta Holes + w_2 \cdot \Delta Area - w_3 \cdot \Delta LinesCleared + K \quad (2.2)$$

Trong đó:

- $\Delta Holes$: Độ thay đổi của số **thành phần liên thông** của không gian trống trên bàn cờ sau khi đặt khối. Giá trị này càng lớn thì không gian trống càng bị chia cắt, gây bất lợi cho các nước đi tiếp theo.
- $\Delta Area$: Độ thay đổi của diện tích **hình chữ nhật nhỏ nhất** bao phủ toàn bộ các ô đã được chiếm trên bàn cờ. Diện tích này càng lớn thì các khối càng bị phân tán và chiếm nhiều không gian hơn.
- $\Delta LinesCleared$: Số lượng hàng hoặc cột được xóa hoàn toàn sau nước đi, giúp làm giảm chi phí.
- w_1, w_2, w_3 : Các trọng số tương ứng cho từng thành phần, được xác định thông qua thực nghiệm.
- K : Hằng số đảm bảo $Cost \geq 0$, thuận tiện cho việc sử dụng trong các thuật toán tìm kiếm có chi phí.

Trong thực tế, nếu số lượng khối trong một lượt tăng lên, chẳng hạn tới 8 khối, thì việc duyệt toàn bộ không gian trạng thái theo kiểu vét cạn sẽ dẫn đến hiện tượng *bùng nổ trạng thái*. Giả sử trung bình mỗi trạng thái có khoảng b hành động hợp lệ và độ sâu tìm kiếm là d , thì số lượng trạng thái cần xét có thể lên tới $O(b^d)$. Vì vậy, nhóm lựa chọn **Best-first Search** thay cho exhaustive search để ưu tiên mở rộng các trạng thái có triển vọng hơn.

2.2 Thuật toán tìm kiếm sử dụng

2.2.1 Best-first Search cho một lượt

Best-first Search là thuật toán tìm kiếm sử dụng hàm đánh giá để xác định thứ tự mở rộng node. Thay vì duyệt toàn bộ cây trạng thái theo chiều rộng hoặc chiều sâu, thuật toán luôn ưu tiên xét trước trạng thái được đánh giá là “tốt” hơn theo một tiêu chí định trước.

Trong bài toán này, mỗi node tương ứng với một trạng thái của một lượt chơi, bao gồm cấu hình bàn cờ hiện tại và danh sách các khối còn lại cần đặt. Các node được lưu trong một hàng đợi ưu tiên (*priority queue*), trong đó node có giá trị đánh giá nhỏ hơn sẽ được mở rộng trước.

Dựa trên hàm chi phí đã xây dựng ở trên, nhóm sử dụng hàm đánh giá cho node dưới dạng:

$$Eval(n) = g(n) + \lambda \cdot |Blocks(n)| \quad (2.3)$$

trong đó:

- $g(n)$ là tổng chi phí tích lũy từ trạng thái đầu đến node n :

$$g(n) = \sum_{i=1}^k Cost(s_{i-1}, a_i, s_i) \quad (2.4)$$

- $|Blocks(n)|$ là số lượng khối còn lại chưa đặt tại node n .
- $\lambda > 0$ là hệ số điều chỉnh nhằm ưu tiên các trạng thái gần hoàn thành lượt chơi hơn.

Như vậy, node có giá trị $Eval(n)$ nhỏ hơn sẽ được xem là hứa hẹn hơn và được đưa ra mở rộng trước. Cách tiếp cận này giúp giảm số lượng trạng thái phải xét so với tìm kiếm vét cạn, đồng thời vẫn hướng thuật toán tới các cấu hình bàn cờ có chất lượng tốt.

Listing 2.2: Giả mã Best-first Search cho một lượt

```

1 BestFirstSearch(initial_state):
2   frontier <- priority queue ordered by Eval(
      node)
3   root <- Node(initial_state)
4   push root into frontier
5
6   visited <- empty map
7   best_terminal <- None
8
9   while frontier is not empty:
10    node <- pop node with smallest Eval(node)
11
12    if Terminal(node.state):
13      if best_terminal is None or
14         node.state.current_score >
           best_terminal.state.
             current_score:
15         best_terminal <- node
16      continue
17
18    key <- Key(node.state)
19
```

```

20         if key in visited and visited[key] <= Eval
           (node):
21             continue
22         visited[key] <- Eval(node)
23
24         for each action in A(node.state):
25             child_state <- Result(node.state,
                                   action)
26             child_node <- Node(
27                 state = child_state,
28                 parent = node,
29                 action = action,
30                 path_cost = g(node) + Cost(node.
                                   state, action, child_state)
31             )
32             push child_node into frontier
33
34         return best_terminal

```

Trong giả mã trên:

- *frontier* là tập các node đang chờ được mở rộng, được tổ chức dưới dạng hàng đợi ưu tiên.
- *visited* dùng để tránh mở rộng lặp lại các trạng thái đã gặp với giá trị đánh giá tốt hơn.
- *best_terminal* lưu trạng thái kết thúc tốt nhất tìm được trong suốt quá trình tìm kiếm.

Ưu điểm của Best-first Search là có thể nhanh chóng tìm được những phương án đặt khối tốt mà không cần duyệt toàn bộ cây tìm kiếm. Tuy nhiên, do thuật toán phụ thuộc mạnh vào hàm đánh giá, chất lượng nghiệm thu được cũng phụ thuộc trực tiếp vào cách lựa chọn các đặc trưng như số thành phần liên thông của vùng trống, mức độ mở rộng diện tích bao phủ và số hàng/cột được xóa.

Chương 3

Thiết kế heuristic cho hệ thống tìm kiếm

3.1 Vai trò của heuristic trong hệ thống

Trong bài toán xếp khối trên bàn cờ, số lượng trạng thái có thể phát sinh tăng nhanh theo số lượng khối cần đặt và số vị trí đặt hợp lệ của mỗi khối. Nếu duyệt toàn bộ không gian trạng thái theo kiểu vét cạn, chi phí tính toán sẽ rất lớn. Vì vậy, hệ thống cần một cơ chế đánh giá để ưu tiên mở rộng những trạng thái có triển vọng hơn.

Trong đề tài này, nhóm sử dụng thuật toán Best-first Search. Thuật toán này hoạt động dựa trên một hàm đánh giá để xác định node nào sẽ được mở rộng trước. Do đó, heuristic đóng vai trò định hướng quá trình tìm kiếm, giúp hệ thống tập trung vào những trạng thái có khả năng dẫn tới kết quả tốt hơn.

3.2 Hàm heuristic được sử dụng

Trong hệ thống đề xuất, giá trị đánh giá của một node được xác định bởi:

$$Eval(n) = g(n) + h(n) \quad (3.1)$$

Trong đó:

- $g(n)$ là tổng chi phí tích lũy từ trạng thái ban đầu đến node n ;
- $h(n)$ là giá trị heuristic ước lượng mức độ còn lại của trạng thái hiện tại.

Chi phí tích lũy được tính bởi:

$$g(n) = \sum_{i=1}^k Cost(s_{i-1}, a_i, s_i) \quad (3.2)$$

với:

$$Cost(s, a, s') = w_1 \cdot \Delta Holes + w_2 \cdot \Delta Area - w_3 \cdot \Delta LinesCleared + K \quad (3.3)$$

Trong đó:

- $\Delta Holes$ là độ thay đổi của số thành phần liên thông của vùng trống;
- $\Delta Area$ là độ thay đổi của diện tích hình chữ nhật nhỏ nhất bao phủ các ô đã chiếm;
- $\Delta LinesCleared$ là số hàng hoặc cột được xóa sau nước đi;
- K là hằng số để đảm bảo chi phí không âm.

Với phần heuristic, nhóm sử dụng số lượng block còn lại trong lượt:

$$h(n) = \lambda \cdot |Blocks(n)| \quad (3.4)$$

Trong đó:

- $|Blocks(n)|$ là số lượng khối còn lại chưa đặt tại node n ;
- $\lambda > 0$ là hệ số điều chỉnh.

Heuristic này được chọn vì phù hợp với mục tiêu của bài toán: trạng thái càng gần hoàn thành lượt chơi thì càng nên được ưu tiên. Khi kết hợp với $g(n)$, hệ thống vừa đánh giá được chất lượng của các nước đi đã thực hiện, vừa ước lượng được mức độ còn lại của bài toán.

Nhận xét

Heuristic được sử dụng trong hệ thống có ưu điểm là đơn giản, dễ tính và phù hợp với Best-first Search. Phần $g(n)$ đã phản ánh nhiều đặc trưng quan trọng của bàn cờ như mức độ phân mảnh của vùng trống, độ dàn trải của các khối và khả năng xóa hàng hoặc cột, trong khi $h(n)$ đóng vai trò ước lượng khoảng cách đến mục tiêu thông qua số block còn lại.

Cách thiết kế này giúp hệ thống giữ được sự cân bằng giữa chất lượng đánh giá và chi phí tính toán. Tuy nhiên, hiệu quả của heuristic vẫn phụ thuộc vào việc lựa chọn các trọng số w_1, w_2, w_3 và hệ số λ . Trong các trường hợp số lượng block tăng lên, hệ thống vẫn có thể gặp hiện tượng bùng nổ trạng thái nếu không giới hạn số node được mở rộng.

3.3 Đánh giá thực nghiệm các trọng số (Ablation Study)

Để đánh giá mức độ đóng góp và sự cần thiết của từng yếu tố trong hàm đánh giá heuristic, nhóm đã thực hiện nghiên cứu cắt bỏ (Ablation Study). Hàm chi phí chuyển trạng thái được sử dụng trong hệ thống có dạng:

$$Cost(s, a, s') = w_1 \cdot \Delta Holes + w_2 \cdot \Delta Area - w_3 \cdot \Delta LinesCleared + K \quad (3.5)$$

Trong đó, mỗi thành phần đại diện cho một chiến lược cụ thể trong việc sắp xếp khối. Nghiên cứu tiến hành so sánh mô hình cơ sở (Baseline) – sử dụng đầy đủ cả ba trọng số – với các kịch bản bị khuyết hoặc thay đổi cực đoan một thành phần trọng số. Các chỉ số được sử dụng để đánh giá bao gồm điểm số trung bình đạt được và số lượt chơi duy trì được trước khi không còn không gian hợp lệ.

3.3.1 Kịch bản Baseline

Kịch bản cơ sở sử dụng bộ trọng số mặc định được thiết lập qua quá trình thử nghiệm ban đầu (ví dụ: $w_1 = 4$, $w_2 = 2$, $w_3 = 6$). Ở thiết lập này, hệ thống thể hiện sự cân bằng giữa việc phòng thủ (giữ cho bàn cờ gọn gàng, ít bị chia cắt) và tấn công (tích cực xóa hàng để ghi điểm). Qua quá trình chạy thử nghiệm, kịch bản Baseline đạt hiệu năng tốt nhất với điểm số trung bình lên tới 1.850 điểm và duy trì lượng không gian hợp lệ trung bình trong khoảng 45 lượt chơi. Đây là mốc chuẩn để đánh giá sự sụt giảm hiệu năng của các kịch bản khác.

3.3.2 Kịch bản 1: Loại bỏ yếu tố kiểm soát phân mảnh ($w_1 = 0$)

Khi thiết lập trọng số $\Delta Holes$ về 0, hệ thống hoàn toàn phớt lờ việc các khoảng trống bị cô lập sinh ra sau mỗi nước đi. Quan sát thực nghiệm cho thấy, thuật toán có xu hướng đặt khối một cách tùy tiện miễn là thỏa mãn các tiêu chí khác. Hậu quả là bàn cờ nhanh chóng xuất hiện các "lỗ hổng" một ô hoặc hai ô xen kẽ nhau. Do kích thước các khối đa dạng, không gian bị chia cắt mạnh khiến hệ thống không thể đặt được các khối lớn ở những lượt tiếp theo. Kết quả là hệ thống bế tắc từ rất sớm, chỉ duy trì trung bình được 12 lượt chơi và điểm số trung bình tụt dốc thảm hại xuống chỉ còn khoảng 210 điểm (giảm gần 88% so với Baseline)

3.3.3 Kịch bản 2: Loại bỏ yếu tố kiểm soát diện tích bao phủ ($w_2 = 0$)

Biến số $\Delta Area$ nhằm hạn chế sự dàn trải của các khối trên bàn cờ. Khi bỏ qua yếu tố này ($w_2 = 0$), hệ thống có xu hướng rải rác khối khắp các góc hoặc cạnh bàn cờ

thay vì tập trung xây dựng các khối đặc. Mặc dù số lượng lỗ hổng có thể không tăng đột biến, nhưng việc các khối nằm cách xa nhau khiến xác suất lấp đầy hoàn toàn một hàng hoặc một cột giảm đi đáng kể. Điều này thể hiện rõ qua số liệu: mặc dù mô hình sống sót tương đối lâu (trung bình 28 lượt chơi), nhưng tổng điểm ghi được lại rất thấp, chỉ đạt khoảng 450 điểm. Hệ thống tuy không "chết" nhanh như kịch bản 1 nhưng hoàn toàn mất đi khả năng tối ưu hóa điểm số

3.3.4 Kịch bản 3: Tối đa hóa cục bộ việc xóa hàng ($w_3 \gg w_1, w_2$)

Trong kịch bản này, trọng số của việc xóa hàng $\Delta LinesCleared$ được đẩy lên cực đại (ví dụ $w_3 = 30$). Thuật toán trở nên "tham lam" (greedy), sẵn sàng thực hiện mọi nước đi để xóa được dù chỉ một hàng ở lượt hiện tại. Sự thiên cận này khiến hệ thống tự phá vỡ các cấu trúc khối đang được xây dựng tốt, tạo ra các lỗ hổng nghiêm trọng chỉ để đổi lấy lợi ích nhỏ trước mắt. Thực nghiệm cho thấy, chiến lược này làm mất đi khả năng thực hiện các chuỗi xóa hàng liên tiếp (combo). Hệ thống bế tắc ở mức trung bình 20 lượt chơi với mức điểm dừng lại ở 680 điểm, kém xa so với sự ổn định dài hạn của kịch bản cơ sở

Nhận xét

Các số liệu thực nghiệm từ quá trình Ablation Study khẳng định rằng không có một thành phần đơn lẻ nào trong hàm heuristic là dư thừa. Sự hiệu quả của hệ thống tìm kiếm (thể hiện qua mức điểm 1.850 của Baseline) phụ thuộc hoàn toàn vào sự tương tác và kìm hãm lẫn nhau giữa ba yếu tố: ngăn chặn phân mảnh (w_1), hạn chế dàn trải (w_2) và động lực ghi điểm (w_3). Việc thiết lập tỷ lệ phù hợp giữa ba trọng số này là điều kiện tiên quyết để đạt được hiệu năng tối ưu trong bài toán sắp xếp khối

Chương 4

Biểu diễn và suy luận tri thức

4.1 Tri thức bổ sung trong hệ thống

Ngoài hàm heuristic định lượng đã trình bày ở chương trước, hệ thống còn được bổ sung một lớp tri thức nhằm hỗ trợ việc lựa chọn giữa các trạng thái có mức đánh giá gần tương đương. Phần tri thức này không thay thế heuristic chính mà chỉ đóng vai trò phụ trợ trong quá trình ra quyết định.

Trong bài toán xếp khối, có một số nhận xét mang tính chiến lược như:

- nước đi xóa được hàng hoặc cột thường là nước đi tốt;
- nước đi làm tăng số thành phần liên thông của vùng trống thường là bất lợi;
- nước đi làm tăng mạnh diện tích bao phủ nhưng không xóa được line nào thường nên tránh.

Những tri thức này có thể được biểu diễn dưới dạng luật để suy ra mức ưu tiên của action. Ví dụ:

- Nếu một action xóa được ít nhất một hàng hoặc cột thì action đó được ưu tiên.
- Nếu một action làm tăng số thành phần liên thông của vùng trống thì action đó bị giảm ưu tiên.
- Nếu một action làm tăng diện tích bao phủ nhưng không xóa được line nào thì action đó bị xem là kém ưu tiên hơn.

Từ các luật trên, hệ thống gán cho mỗi action hoặc node một điểm phụ, gọi là *RuleScore*, phản ánh mức độ phù hợp về mặt chiến lược.

4.2 Cơ chế tie-break trong hệ thống

Trong hệ thống hiện tại, hàm đánh giá chính vẫn được giữ nguyên:

$$Eval(n) = g(n) + h(n) \quad (4.1)$$

Tri thức ở chương này không được đưa trực tiếp vào công thức trên, mà chỉ được dùng để hỗ trợ phân xử khi các node có giá trị đánh giá bằng nhau hoặc gần bằng nhau.

Ý tưởng của cơ chế tie-break là:

- nếu một node có giá trị *Eval* nhỏ hơn thì node đó vẫn được ưu tiên trước;
- nếu hai node có giá trị *Eval* bằng nhau, hoặc chênh lệch rất nhỏ, thì hệ thống xét thêm *RuleScore*;
- node nào có *RuleScore* cao hơn sẽ được ưu tiên mở rộng trước.

Có thể mô tả nguyên tắc này như sau:

$$\begin{aligned} n_1 \prec n_2 \iff & (Eval(n_1) < Eval(n_2)) \\ & \vee (Eval(n_1) = Eval(n_2) \wedge RuleScore(n_1) > RuleScore(n_2)) \end{aligned} \quad (4.2)$$

Như vậy, heuristic vẫn là tiêu chí chính, còn tri thức chỉ là tiêu chí phụ để phân xử trong những trường hợp khó phân biệt bằng các giá trị số học.

Nhận xét

Cách bổ sung tri thức theo cơ chế tie-break có ưu điểm là không làm thay đổi cấu trúc chính của thuật toán Best-first Search, đồng thời vẫn đáp ứng được yêu cầu về biểu diễn và suy luận tri thức của hệ thống. Nhờ đó, hệ thống có thể tận dụng thêm các kinh nghiệm chiến lược mà không cần thay thế hoàn toàn hàm heuristic đã xây dựng trước đó.

Bên cạnh đó, cơ chế này cũng dễ mở rộng: nếu sau này cần bổ sung thêm tri thức mới, nhóm chỉ cần thêm các luật tương ứng để điều chỉnh *RuleScore* mà không phải thiết kế lại toàn bộ hàm đánh giá chính. Tuy nhiên, chất lượng tie-break vẫn phụ thuộc vào việc lựa chọn tập luật có hợp lý hay không. Nếu các luật quá đơn giản, mức cải thiện đạt được sẽ không lớn.

Chương 5

Mạng Bayes và suy diễn xác suất rủi ro

5.1 Mục tiêu của chương

Trong các chương trước, hệ thống đã có thuật toán tìm kiếm và heuristic để chọn nước đi. Tuy nhiên, trong Block Blast vẫn tồn tại yếu tố không chắc chắn: một nước đi có thể đem lại lợi ích ngắn hạn, ví dụ xóa line, nhưng lại làm bàn cờ trở nên bất lợi ở các bước kế tiếp.

Vì vậy, mục tiêu của chương này là mô hình hóa rủi ro bị kẹt sau một action bằng Mạng Bayes. Cụ thể:

- Chọn yếu tố không chắc chắn là rủi ro bị kẹt sau một action, được biểu diễn bởi biến StuckRisk với ba mức Low, Medium, High.
- Sử dụng Mạng Bayes để suy diễn từ các bằng chứng quan sát được trên board.
- Tính giá trị $P(\text{StuckRisk} = \text{High})$ để hỗ trợ xếp hạng action theo mức an toàn.

5.2 Cơ sở lý thuyết

Mạng Bayes, hay Bayesian Network, là một đồ thị có hướng không chu trình, trong đó:

- Mỗi đỉnh biểu diễn một biến ngẫu nhiên.
- Mỗi cạnh biểu diễn ảnh hưởng trực tiếp giữa hai biến.

- Mỗi biến có một bảng xác suất có điều kiện, gọi là CPD.

Với mỗi hàng trong CPD, tổng các xác suất phải bằng 1. Mục tiêu của suy diễn Bayes là tính phân phối hậu nghiệm của biến cần quan tâm khi đã biết bằng chứng.

5.3 Định nghĩa các biến ngẫu nhiên

Trong module Bayes của hệ thống, các biến được định nghĩa như Bảng 5.1.

Bảng 5.1: Các biến ngẫu nhiên trong mô hình Bayes

Biến	Ý nghĩa	Miền giá trị
Density	Mức độ đầy của board sau action	Low, Medium, High
Fragmentation	Mức phân mảnh vùng trống sau action	Low, Medium, High
LineClear	Action có xóa hàng/cột hay không	Yes, No
ValidMoveLevel	Mức nhiều/ít action hợp lệ còn lại đối với các block chưa đặt trong lượt hiện tại sau khi thực hiện action	Low, Medium, High
StuckRisk	Mức rủi ro bị kẹt được đo từ kết quả rollout mô phỏng sau action	Low, Medium, High

Trong mô hình này, Density, Fragmentation và LineClear là các bằng chứng quan sát được sau khi thử một action.

ValidMoveLevel là biến trung gian biểu diễn mức độ còn nước đi hợp lệ sau khi thực hiện một action. Trong phạm vi bài toán một lượt, biến này không được tính trên toàn bộ các loại Tetriminos, mà được tính trên danh sách các block còn lại chưa đặt trong lượt hiện tại. Vì vậy, ValidMoveLevel phản ánh khả năng tiếp tục hoàn thành lượt chơi sau action đang xét.

StuckRisk là biến cần suy diễn, biểu diễn rủi ro trạng thái sau action dẫn đến tình huống khó tiếp tục đặt block. Trong thực nghiệm, nhân StuckRisk không được gán trực tiếp bằng luật thủ công, mà được xác định thông qua rollout mô phỏng: sau khi thực hiện action, hệ thống tiếp tục chơi một số bước/lượt bằng chính cơ chế mô phỏng, sau đó dựa trên kết quả có bị kẹt sớm hay không để gán mức Low, Medium hoặc High.

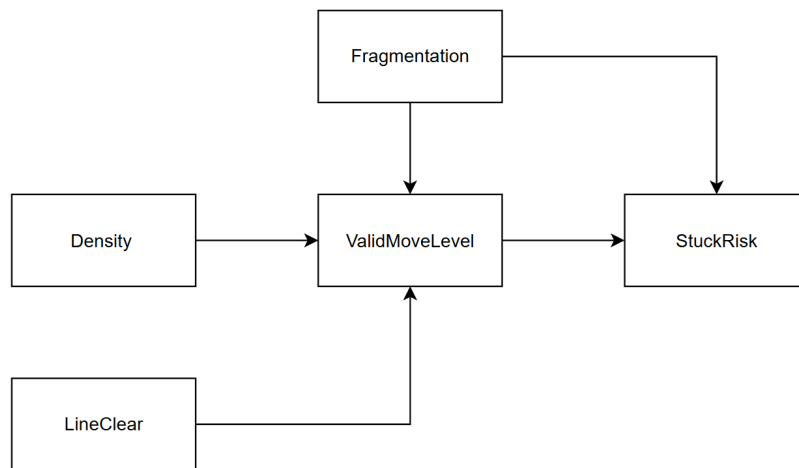
5.4 Phạm vi thông tin

Trong chương này, các xác suất trong CPD được ước lượng từ dữ liệu mô phỏng thay vì gán thủ công. Mỗi mẫu dữ liệu tương ứng với một action hợp lệ trong môi trường Block Blast mô phỏng. Sau khi action được thực hiện, hệ thống ghi nhận các đặc trưng Density, Fragmentation, LineClear và ValidMoveLevel. Sau đó, hệ thống tiếp tục rollout trong môi trường mô phỏng để xác định mức StuckRisk của trạng thái kế tiếp.

Vì vậy, các xác suất trong chương này được hiểu là xác suất thực nghiệm trong môi trường mô phỏng của hệ thống, không phải xác suất từ người chơi thật ngoài đời. Tuy nhiên, do toàn bộ trò chơi trong đề tài được xây dựng bằng mô phỏng, đây vẫn là dữ liệu thực nghiệm phù hợp với phạm vi bài toán.

5.5 Cấu trúc mạng Bayes

Cấu trúc DAG được giả định như Hình 5.1.



Hình 5.1: Đồ thị có hướng không chu trình của mô hình Bayes

Ý nghĩa các cạnh:

- Density → ValidMoveLevel: board càng đầy thì số nước đi hợp lệ kế tiếp thường giảm.
- Fragmentation → ValidMoveLevel: vùng trống bị chia cắt làm giảm khả năng đặt block.

- LineClear $\rightarrow$ ValidMoveLevel: xóa line giúp giải phóng không gian, tăng cơ hội đặt block.
- ValidMoveLevel $\rightarrow$ StuckRisk: càng ít nước đi hợp lệ thì rủi ro bị kẹt càng cao.
- Fragmentation $\rightarrow$ StuckRisk: phân mảnh cao làm tăng rủi ro bị kẹt dài hạn.

5.6 Bảng xác suất có điều kiện CPD

Các bảng CPD được ước lượng bằng tần suất thực nghiệm từ tập dữ liệu mô phỏng gồm 50,000 mẫu. Với mỗi tổ hợp giá trị của các biến cha, nếu tổ hợp đó xuất hiện trong dữ liệu, xác suất của biến con được tính bằng tỉ lệ xuất hiện của từng giá trị biến con trên tổng số mẫu thuộc tổ hợp cha đó.

Nhóm không sử dụng Laplace smoothing trong quá trình ước lượng CPD. Do đó, các tổ hợp cha không xuất hiện trong dữ liệu mô phỏng sẽ không được gán xác suất giả định. Các dòng này được đánh dấu là Not observed và không được sử dụng trực tiếp trong suy diễn xác suất.

Cách làm này giúp bảng CPD phản ánh đúng dữ liệu mô phỏng thu được, tránh việc tạo ra các xác suất không có bằng chứng thực nghiệm.

5.7 Suy diễn xác suất

Trong mạng Bayes được xây dựng, các biến Density, Fragmentation và LineClear là các biến quan sát được sau khi xét một action cụ thể. Từ các thông tin này, hệ thống suy diễn xác suất của biến StuckRisk, tức mức rủi ro bị kẹt sau khi thực hiện action.

Để công thức gọn hơn, ký hiệu:

$D = \text{Density}$, $F = \text{Fragmentation}$, $L = \text{LineClear}$, $V = \text{ValidMoveLevel}$, $R = \text{StuckRisk}$.

Các CPD trong mạng được ước lượng từ dữ liệu mô phỏng bằng tần suất thực nghiệm:

$$\hat{P}(X = x \mid \text{Pa}(X) = p) = \frac{N(X = x, \text{Pa}(X) = p)}{N(\text{Pa}(X) = p)}, \quad N(\text{Pa}(X) = p) > 0.$$

Nhóm không sử dụng Laplace smoothing. Vì vậy, nếu một tổ hợp giá trị của các biến cha không xuất hiện trong dữ liệu mô phỏng, hệ thống không tự gán xác suất cho tổ hợp đó. Trường hợp này được xem là không đủ dữ liệu để suy diễn.

Do ValidMoveLevel là biến trung gian, xác suất rủi ro được tính bằng cách lấy tổng theo các giá trị có thể có của V :

$$\hat{P}(R = r \mid D = d, F = f, L = l) = \sum_{v \in \{\text{Low}, \text{Medium}, \text{High}\}} \hat{P}(R = r \mid V = v, F = f) \cdot \hat{P}(V = v \mid D = d, F = f, L = l).$$

Trong phạm vi hệ thống, giá trị quan trọng nhất để hỗ trợ đánh giá action là:

$$\hat{P}(R = \text{High} \mid D = d, F = f, L = l).$$

Action nào có xác suất rủi ro cao lớn hơn sẽ được xem là có nguy cơ làm bàn cờ rơi vào trạng thái khó đặt khối hơn trong các bước tiếp theo. Giá trị này không thay thế hoàn toàn heuristic của hệ thống tìm kiếm, mà đóng vai trò bổ sung thêm một tiêu chí xác suất khi đánh giá action.

5.8 Kết quả thực nghiệm của module Bayes

Module Bayes được đánh giá trên tập dữ liệu mô phỏng sinh ra từ môi trường Block Blast. Mỗi mẫu dữ liệu tương ứng với một action hợp lệ. Sau khi action được thực hiện, hệ thống ghi nhận các biến D , F , L , V và tiếp tục mô phỏng để xác định nhãn R .

Bảng 5.2: Phân phối nhãn StuckRisk trong dữ liệu mô phỏng

StuckRisk	Số mẫu	Tỉ lệ
Low	15856	31.712%
Medium	21225	42.450%
High	12919	25.838%

Kết quả cho thấy phần lớn các trạng thái sau action rơi vào mức rủi ro trung bình hoặc thấp. Cụ thể, nhóm Medium chiếm tỉ lệ cao nhất với 42.450%, tiếp theo là Low với 31.712% và High với 25.838%. Điều này cho thấy tập dữ liệu mô phỏng không bị lệch hoàn toàn về một nhãn duy nhất, nên có thể dùng để ước lượng các CPD phục vụ suy diễn xác suất rủi ro.

Do không sử dụng smoothing, độ phủ dữ liệu là yếu tố quan trọng cần kiểm tra. Nếu một tổ hợp giá trị của các biến cha không xuất hiện trong dữ liệu mô phỏng, hệ thống không suy diễn xác suất cho tổ hợp đó.

Bảng 5.3: Độ phủ dữ liệu của các bảng CPD

CPD	Tổng tổ hợp lý thuyết	Số tổ hợp quan sát được	Số tổ hợp chưa quan sát	Tỉ lệ coverage
$P(V D, F, L)$	18	15	3	83.333%
$P(R V, F)$	9	9	0	100.000%

Bảng 5.3 cho thấy CPD của StuckRisk có độ phủ đầy đủ trên toàn bộ 9 tổ hợp lý thuyết của hai biến cha ValidMoveLevel và Fragmentation. Trong khi đó, CPD của ValidMoveLevel quan sát được 15 trên 18 tổ hợp lý thuyết, tương ứng 83.333% độ phủ. Ba tổ hợp còn lại không xuất hiện trong 50,000 mẫu mô phỏng, nên hệ thống không gán xác suất cho các tổ hợp này.

Do nhóm không sử dụng Laplace smoothing, các tổ hợp không quan sát được không được tự động thay bằng xác suất giả định. Khi gặp các tổ hợp này trong suy diễn, module Bayes trả về trạng thái không đủ dữ liệu và hệ thống có thể tiếp tục đánh giá action bằng các tiêu chí heuristic khác.

5.9 Vai trò của module Bayes trong hệ thống

Module Bayes được thiết kế như một thành phần độc lập để đánh giá rủi ro xác suất của action:

- Không thay thế Best-first Search.
- Không thay thế module Decision Tree.
- Bổ sung một góc nhìn xác suất để đánh giá action.
- Có thể dùng $\hat{P}(\text{StuckRisk} = \text{High})$ làm hệ số phạt nhẹ khi cần tích hợp ở mức chiến lược.
- Chỉ suy diễn trên các tổ hợp evidence đã có dữ liệu mô phỏng hỗ trợ.

Nhờ đó, hệ thống vừa giữ được tính diễn giải của heuristic và tìm kiếm, vừa có thêm khả năng lượng hóa rủi ro ngắn hạn và trung hạn theo xác suất thực nghiệm trong môi trường mô phỏng.

5.10 Độ phủ dữ liệu và hướng mở rộng

Trong phiên bản hiện tại, CPD được ước lượng từ dữ liệu mô phỏng thay vì gán thủ công bằng giả định chuyên gia. Quy trình thực nghiệm gồm:

- Sinh các trạng thái bàn cờ trong môi trường mô phỏng.
- Sinh một lượt chơi gồm các block cần đặt.
- Chọn các action hợp lệ và thực hiện action trên bàn cờ.
- Ghi nhận các biến Density, Fragmentation, LineClear và ValidMoveLevel sau action.
- Tiếp tục mô phỏng các bước sau action để xác định nhãn StuckRisk.
- Ước lượng CPD bằng tần suất có điều kiện từ dữ liệu thu được.

Do không sử dụng Laplace smoothing, một số tổ hợp giá trị của các biến cha có thể không xuất hiện trong tập dữ liệu mô phỏng. Đây không được xem là lỗi của mô hình, mà là giới hạn về độ phủ dữ liệu. Với các tổ hợp này, hệ thống không đưa ra xác suất thực nghiệm và đánh dấu là không đủ dữ liệu.

Các hướng mở rộng hợp lý gồm tăng số lượng mẫu mô phỏng, chạy nhiều seed ngẫu nhiên khác nhau, sinh thêm dữ liệu ở các trạng thái hiếm, kiểm tra riêng các tổ hợp nghi ngờ không thể xảy ra theo luật game, và so sánh xác suất Bayes với kết quả chơi thực tế trong mô phỏng. Nếu có dữ liệu từ người chơi thật, các CPD cũng có thể được hiệu chỉnh lại để phản ánh hành vi chơi thực tế tốt hơn.

Trong lần chạy thực nghiệm với 50,000 mẫu, CPD $P(\text{StuckRisk} \mid \text{ValidMoveLevel}, \text{Fragmentation})$ đạt độ phủ 100%, trong khi CPD $P(\text{ValidMoveLevel} \mid \text{Density}, \text{Fragmentation}, \text{LineClear})$ đạt độ phủ 83.333%. Các tổ hợp chưa quan sát được không được gán xác suất do nhóm không sử dụng smoothing. Đây là giới hạn của dữ liệu mô phỏng hiện tại và có thể được cải thiện bằng cách tăng số mẫu hoặc sinh thêm dữ liệu có kiểm soát ở các trạng thái hiếm (hoặc không xảy ra).

Ba tổ hợp không quan sát được đều thuộc nhóm Density = High nhưng Fragmentation thấp/trung bình, trong đó hai tổ hợp còn yêu cầu LineClear = Yes. Về mặt ngữ nghĩa, bàn rất đầy thường làm các ô trống bị chia cắt thành nhiều vùng nhỏ, nên Fragmentation thường cao. Nếu action có xóa dòng, mật độ bàn lại có xu hướng giảm, khiến trạng thái sau action khó còn nằm ở Density = High. Vì vậy các tổ hợp này rất hiếm trong mô phỏng 50,000 mẫu. Nhóm không dùng smoothing nên không tự gán xác suất cho chúng.

Chương 6

Học máy

6.1 Vai trò của học máy trong hệ thống

Trong các chương trước, hệ thống đã sử dụng thuật toán tìm kiếm *Best-first Search* kết hợp với hàm *heuristic* và các vị từ suy luận tri thức để lựa chọn cách đặt block. Tuy nhiên, khi số lượng trạng thái và hành động hợp lệ tăng lên, việc đánh giá toàn bộ các nhánh tìm kiếm bằng heuristic đơn thuần tiêu tốn nhiều tài nguyên tính toán.

Vì vậy, trong chương này, nhóm bổ sung một module học máy đóng vai trò như một **bộ lọc hành động thông minh**. Thay vì phải mở rộng đều mọi hành động, mô hình học máy dự đoán trước xác suất một hành động là tốt ($P(\text{good} \mid \text{state}, \text{action})$) và tích hợp kết quả này vào hàm ưu tiên của Best-first Search. Nhờ đó, hệ thống ưu tiên mở rộng các nhánh hứa hẹn hơn và bỏ qua sớm các nhánh kém tiềm năng.

Bài toán học máy được xây dựng dưới dạng **phân loại nhị phân**. Mỗi mẫu dữ liệu là một cặp $(\text{state}, \text{action})$, trong đó *state* là trạng thái bàn cờ 8×8 hiện tại và *action* là một hành động đặt block hợp lệ. Mô hình dự đoán nhãn:

- **1** – hành động tốt (nằm trong top theo đánh giá của Teacher Signal);
- **0** – hành động không tốt.

Học máy không thay thế thuật toán tìm kiếm mà đóng vai trò *hỗ trợ*: kết quả dự đoán được cộng vào hàm ưu tiên của BFS dưới dạng bonus, giúp điều hướng tìm kiếm hiệu quả hơn.

6.2 Sinh dữ liệu huấn luyện bằng Teacher Signal

Do bài toán không có sẵn tập dữ liệu ngoài, nhóm tự sinh dữ liệu mô phỏng theo cơ chế **Teacher Signal**: dùng BFS có giới hạn làm “giáo viên” để gán nhãn cho từng hành động.

6.2.1 Sinh trạng thái hợp lệ

Hàm `generate_random_reachable_state()` tạo trạng thái bàn cờ thực tế bằng cách:

1. Bắt đầu từ bàn cờ rỗng 8×8 .
2. Thực hiện ngẫu nhiên 2–6 bước đặt block hợp lệ.
3. Chọn ngẫu nhiên 4 block từ tập 25 biến thể Tetrimino cho lượt hiện tại.

Cách này đảm bảo các trạng thái sinh ra phản ánh tình huống thực tế trong game, thay vì chỉ là bàn cờ rỗng hoặc bàn cờ ngẫu nhiên không có ý nghĩa.

6.2.2 Gán nhãn bằng BFS Oracle

Với mỗi trạng thái, nhóm lấy mẫu tối đa `MAX_ACTIONS` hành động hợp lệ. Với mỗi hành động a :

1. Áp dụng a lên trạng thái hiện tại, thu được trạng thái con s' .
2. Chạy Best-first Search từ s' với ngân sách `TEACHER_BUDGET` node mở rộng.
3. Ghi lại điểm tốt nhất BFS đạt được: `teacher_score(a)`.
4. Xếp hạng tất cả hành động trong cùng trạng thái theo `teacher_score`.
5. Gán nhãn 1 cho top `POSITIVE_RATIO` hành động; còn lại gán nhãn 0.

Chiến lược gán nhãn này có tính *tương đối trong cùng trạng thái*: một hành động được gọi là “tốt” khi nó tốt hơn đáng kể so với các hành động thay thế trong cùng tình huống, không phải so với một ngưỡng tuyệt đối cố định. Điều này giúp nhãn có ý nghĩa nhất quán bất kể trạng thái bàn cờ đang ở mức điểm nào.

6.2.3 Cấu hình sinh dữ liệu

Bảng 6.1: Siêu tham số sinh dữ liệu huấn luyện

Tham số	Giá trị	Ý nghĩa
<code>N_STATES</code>	500	Số trạng thái bàn cờ dùng để sinh mẫu
<code>TEACHER_BUDGET</code>	1000	Số node BFS oracle mở rộng mỗi lượt đánh giá
<code>POSITIVE_RATIO</code>	0.15	Tỉ lệ hành động được gán nhãn 1 (top 15%)
<code>MAX_ACTIONS</code>	15	Số hành động tối đa lấy mẫu mỗi trạng thái

Kết quả: tập dữ liệu $\approx 7,497$ mẫu, phân phối nhãn **80% nhãn 0** (xấu) và **20% nhãn 1** (tốt), phản ánh đúng thực tế rằng phần lớn hành động hợp lệ không phải là nước đi tốt nhất.

6.3 Trích xuất đặc trưng

Mỗi cặp (*state, action*) được chuyển thành một vector đặc trưng số thực trước khi đưa vào mô hình. Tổng cộng có **26 đặc trưng gốc** (cộng thêm các cột one-hot từ *block_name*, tổng ≈ 47 chiều sau mã hóa).

Bảng 6.2: Danh sách đặc trưng trích xuất

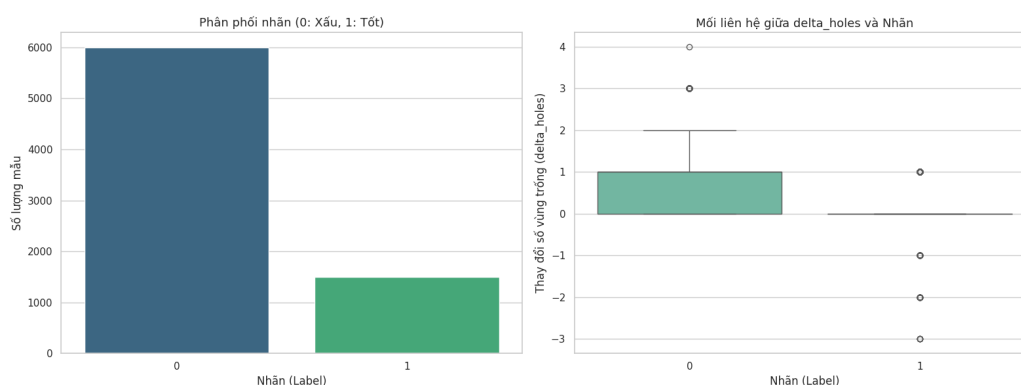
Đặc trưng	Ý nghĩa
<i>Trạng thái bàn cờ</i>	
filled_ratio	Tỉ lệ ô đã lấp đầy trên bàn cờ (0–1)
holes_before/after	Số thành phần liên thông vùng trống trước/sau đặt block
delta_holes	Độ thay đổi số vùng trống: holes_after – holes_before
bbox_before/after	Diện tích bounding box của các ô đã chiếm trước/sau
delta_area	Độ thay đổi diện tích bounding box
<i>Tính linh hoạt (mobility)</i>	
valid_actions_before/after	Số nước đi hợp lệ trước/sau hành động
<i>Tiềm năng xóa hàng/cột</i>	
near_rows/cols_before/after	Số hàng/cột gần đây (≥ 7 ô) trước/sau
lines_cleared	Số hàng hoặc cột được xóa ngay sau hành động
score_gain	Điểm tăng thêm sau hành động
<i>Thông tin block và vị trí</i>	
block_cells	Số ô của block
block_h, block_w	Chiều cao và chiều rộng của block
x_norm, y_norm	Tọa độ đặt block được chuẩn hóa về [0, 1]
block_name	Loại block (one-hot encode thành ~ 21 cột)

Sau khi one-hot encode *block_name*, mỗi mẫu có 47 đặc trưng. Vector đặc trưng toàn bộ tập dữ liệu được lưu vào file `feature_vectors.npy` để phục vụ tái sử dụng.

6.4 EDA - Phân tích khám phá dữ liệu

Tập dữ liệu huấn luyện của hệ thống bao gồm khoảng 7,497 mẫu (đại diện cho các hành động hợp lệ), được thu thập tự động thông qua quá trình tự chơi (self-play) trên 500 trạng thái bàn cờ khác nhau. Tuy nhiên, để đảm bảo chất lượng đầu vào cho mô hình học máy, nhóm đã tiến hành Phân tích Khám phá Dữ liệu (EDA).

6.4.1 Phân phối nhãn và Quan hệ của delta_holes với nhãn



Hình 6.1: Phân phối số lượng mẫu theo Nhãn và mối liên hệ với delta_hole

Số lượng nước đi Xấu (Nhãn 0) chiếm khoảng 80% (gần 6000 mẫu), nước đi Tốt (Nhãn 1) chỉ chiếm 20% (khoảng 1500 mẫu), đáp ứng tốt yêu cầu

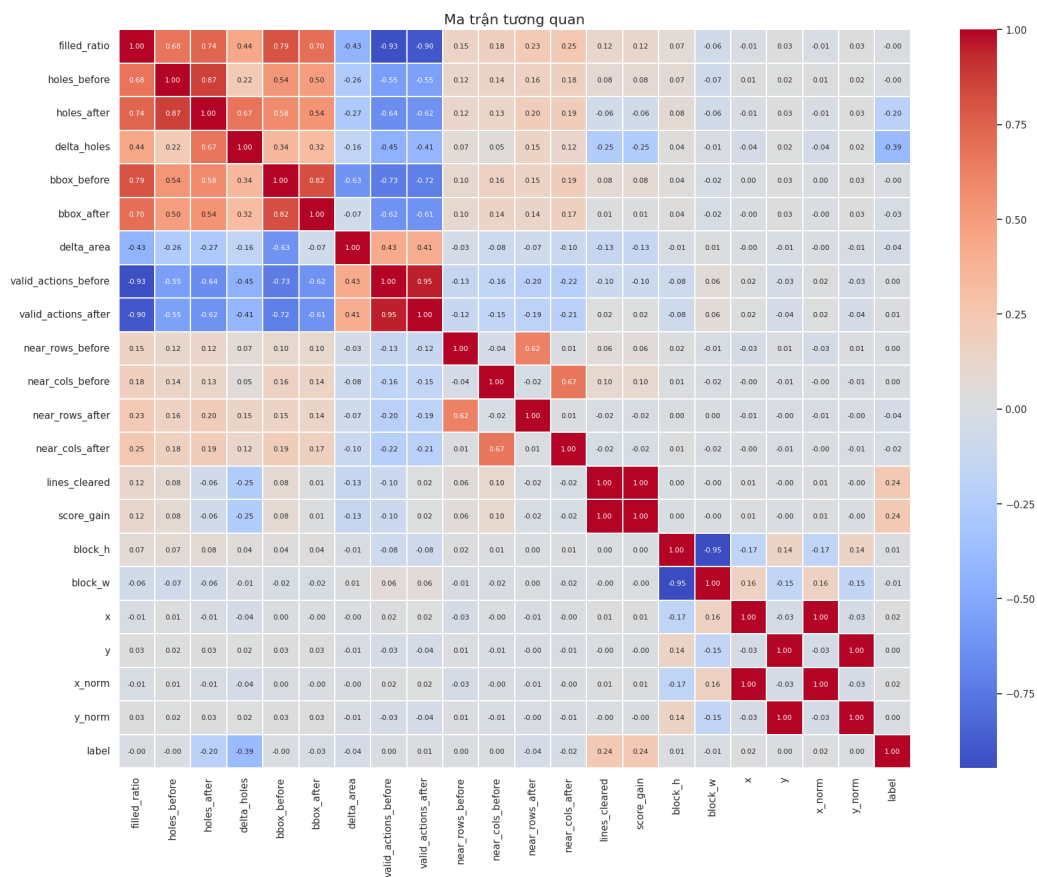
Về mối liên hệ với delta_holes, dataset phản ánh đúng trực giác của người chơi: "Trong Block Blast, kỳ nhất là tạo ra lỗ hổng. Nếu 1 nước đi thêm lỗ vào bàn cờ ($\text{delta_holes} > 0$) thì nó là nước đi xấu, và nếu nước nào không tạo ra lỗ, hoặc (hiếm) giảm số lỗ, nước đó sẽ là nước tốt.

6.4.2 Ma trận đặc trưng

Theo ma trận 6.2 các đặc trưng có độ tương quan bằng 1:

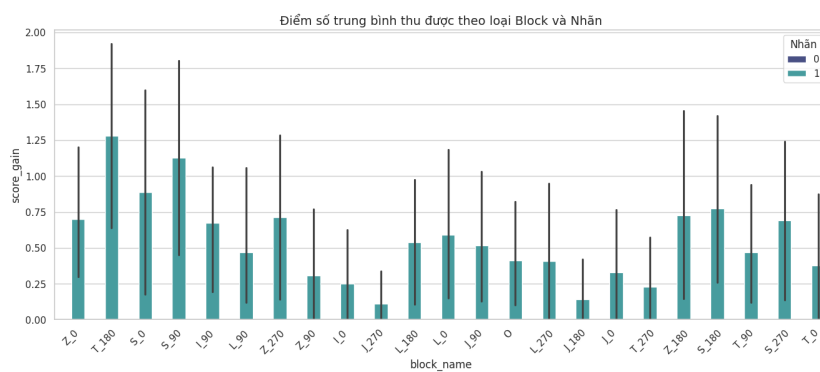
- *score_gain* và *lines_cleared*
- *x* và *x_norm*
- *y* và *y_norm*

Vì thế, trước khi đưa vào mô hình, chúng ta cần loại bỏ đặc trưng *score_gain*, *x* và *y*. Ngoài ra, dựa theo ma trận, có thể thấy delta_holes là biến có mức độ ảnh hưởng tới label là cao nhất.



Hình 6.2: Ma trận đặc trưng

6.4.3 Boxplot: điểm số trung bình theo nhãn



Hình 6.3: Điểm số trung bình theo nhãn

Biểu đồ Boxplot 6.3, ta có thể thấy một điều đặc biệt, không hề có một cột nào mà nhãn 0 (màu xanh đậm) xuất hiện. Điều này cho thấy Teacher Signal nhận định rằng không có nước đi nào ăn điểm lại là nước đi xấu cả. Thiên kiến này phản ánh tốt ý thức của người chơi.

6.5 Mô hình học máy

Nhóm huấn luyện và so sánh hai mô hình phân loại:

6.5.1 Decision Tree Classifier

Decision Tree chia không gian đặc trưng thành các vùng bằng chuỗi điều kiện ngưỡng. Mô hình phù hợp với bài toán này vì:

- Các đặc trưng mang tính ngưỡng rõ ràng (ví dụ: $\text{delta_holes} \leq 0.5$ ứng với hành động không tạo thêm vùng cô lập).
- Có thể trích xuất luật giải thích được (interpretable rules).
- Tốc độ suy diễn nhanh, phù hợp khi tích hợp vào vòng lặp BFS.

Cấu hình huấn luyện:

Bảng 6.3: Siêu tham số Decision Tree

Tham số	Giá trị	Ý nghĩa
max_depth	6	Độ sâu tối đa của cây
min_samples_leaf	8	Số mẫu tối thiểu tại nút lá
class_weight	balanced	Bù mất cân bằng nhãn tự động
random_state	42	Cố định kết quả tái lập

6.5.2 Random Forest Classifier

Random Forest là tập hợp 100 Decision Tree độc lập, mỗi cây được huấn luyện trên bootstrap sample và chỉ xét $\sqrt{n_{\text{features}}}$ đặc trưng ngẫu nhiên tại mỗi bước phân chia. Ưu điểm:

- Giảm variance so với một cây đơn lẻ.
- Ít nhạy cảm với feature dominant (mỗi cây chỉ xét một tập con đặc trưng).
- Phân phối feature importance phân tán đều hơn.

Cấu hình huấn luyện:

Bảng 6.4: Siêu tham số Random Forest

Tham số	Giá trị	Ý nghĩa
n_estimators	100	Số cây trong rừng
max_depth	8	Độ sâu tối đa mỗi cây
min_samples_leaf	5	Số mẫu tối thiểu tại nút lá
max_features	sqrt	Số đặc trưng xét tại mỗi split
class_weight	balanced	Bù mất cân bằng nhãn tự động

6.6 Huấn luyện và đánh giá mô hình

6.6.1 Phân chia dữ liệu

Tập dữ liệu 7,497 mẫu được chia theo tỉ lệ **80/20** (stratified) để giữ nguyên tỉ lệ nhãn trong cả tập train và test:

- Tập huấn luyện: 5,997 mẫu.
- Tập kiểm tra: 1,500 mẫu (1,200 nhãn 0 và 300 nhãn 1).

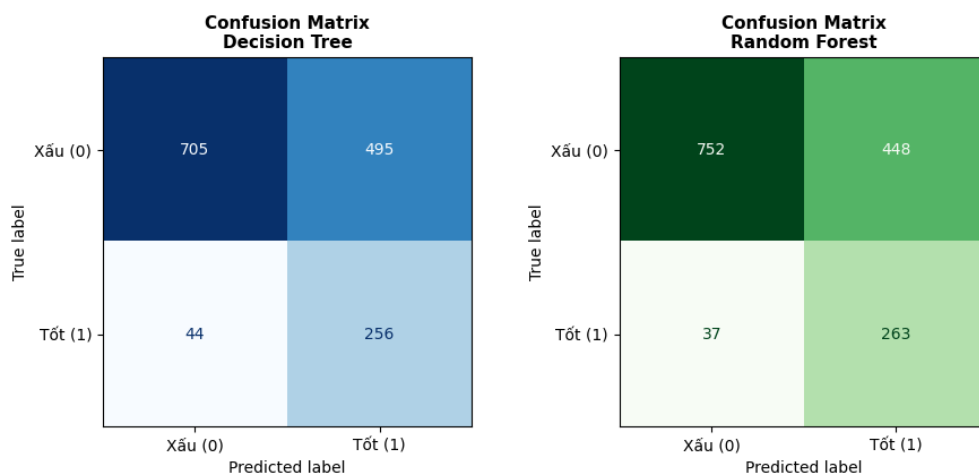
6.6.2 Kết quả trên tập kiểm tra

Bảng 6.5: Kết quả đánh giá Decision Tree trên tập test

Lớp	Precision	Recall	F1-score	Support
Xấu (0)	0.94	0.59	0.72	1200
Tốt (1)	0.34	0.85	0.49	300
<i>Accuracy</i>	-	-	0.641	1500
<i>Macro avg</i>	0.64	0.72	0.61	1500
<i>Weighted avg</i>	0.82	0.64	0.68	1500

Bảng 6.6: Kết quả đánh giá Random Forest trên tập test

Lớp	Precision	Recall	F1-score	Support
Xấu (0)	0.95	0.63	0.76	1200
Tốt (1)	0.37	0.88	0.52	300
<i>Accuracy</i>	-	-	0.677	1500
<i>Macro avg</i>	0.66	0.75	0.64	1500
<i>Weighted avg</i>	0.84	0.68	0.71	1500



Hình 6.4: Confusion Matrix so sánh Decision Tree và Random Forest trên tập test

6.6.3 Đánh giá bằng 5-fold Cross-Validation

Để đảm bảo kết quả không phụ thuộc vào một lần phân chia ngẫu nhiên, nhóm thực hiện **Stratified 5-fold Cross-Validation**: toàn bộ dữ liệu được chia thành 5 fold, mỗi lần train trên 4 fold và kiểm tra trên fold còn lại. Kết quả:

```
... Đang chạy 5-fold cross-validation...
(mỗi fold train 80% test 20%, xoay vòng 5 lần)

— Decision Tree —
accuracy : 0.655 ± 0.013 (folds: 0.67 0.65 0.64 0.65 0.67)
f1       : 0.502 ± 0.004 (folds: 0.51 0.50 0.50 0.50 0.50)
precision : 0.353 ± 0.007 (folds: 0.36 0.35 0.34 0.35 0.36)
recall    : 0.869 ± 0.025 (folds: 0.85 0.86 0.91 0.89 0.84)

— Random Forest —
accuracy : 0.665 ± 0.007 (folds: 0.68 0.67 0.65 0.66 0.67)
f1       : 0.504 ± 0.005 (folds: 0.51 0.51 0.50 0.50 0.51)
precision : 0.358 ± 0.005 (folds: 0.36 0.36 0.35 0.35 0.36)
recall    : 0.851 ± 0.016 (folds: 0.84 0.87 0.86 0.83 0.86)
```

Hình 6.5: Stratified 5-fold Cross-Validation

Nhận xét: F1-score rất ổn định qua cả 5 fold (độ lệch chuẩn ≤ 0.005), cho thấy mô hình không bị overfitting và kết quả có tính tổng quát tốt.

6.6.4 Tầm quan trọng của đặc trưng

```
... =====
TOP 10 FEATURES – Decision Tree
=====
```

Đặc trưng	Importance
delta_holes	0.740496
valid_actions_before	0.083330
filled_ratio	0.081017
valid_actions_after	0.016053
bbox_after	0.013744
y	0.011676
block_name_I_0	0.007284
lines_cleared	0.007026
score_gain	0.005166
block_name_J_90	0.004913

```
=====
TOP 10 FEATURES – Random Forest
=====
```

Đặc trưng	Importance
delta_holes	0.459151
holes_after	0.109513
score_gain	0.054116
valid_actions_before	0.048842
valid_actions_after	0.043978
lines_cleared	0.038498
holes_before	0.036435
filled_ratio	0.026924
delta_area	0.024414
bbox_before	0.021827

Hình 6.6: Tầm quan trọng của đặc trưng theo hai mô hình

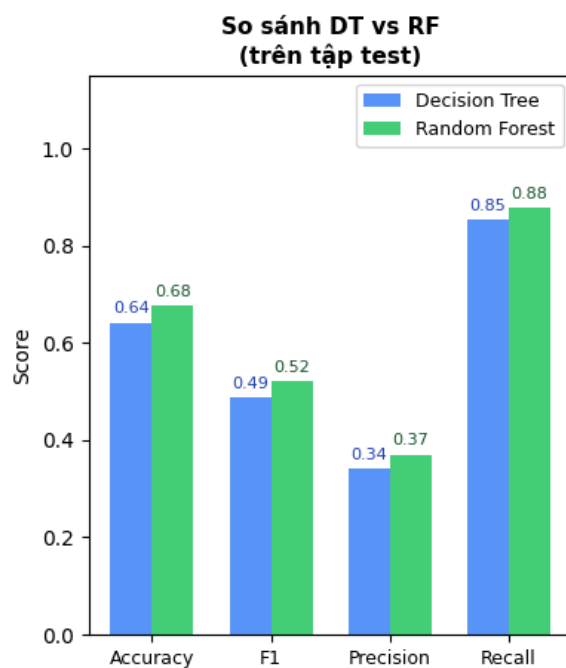
Phân tích: Đặc trưng `delta_holes` chiếm tỉ trọng lớn nhất ở cả hai mô hình (74% với DT, 46% với RF). Điều này phản ánh đúng thực tế: số vùng trống cô lập tăng sau một hành động là dấu hiệu rõ ràng nhất của nước đi xấu. Random Forest phân bổ tỉ trọng đều hơn, khai thác thêm `holes_after`, `score_gain`, `valid_actions_before/after` – cho thấy RF học được các tương tác phức tạp hơn giữa các đặc trưng.

6.7 So sánh và lựa chọn mô hình

6.7.1 So sánh tổng quan

Bảng 6.7: Bảng so sánh Decision Tree và Random Forest

Tiêu chí	Decision Tree	Random Forest
Accuracy (test)	0.641	0.677
F1-score (test)	0.487	0.520
F1-score (CV mean)	0.502	0.504
Precision (lớp Tốt)	0.34	0.37
Recall (lớp Tốt)	0.85	0.88
Tốc độ suy diễn	Rất nhanh	Chậm hơn (×100 cây)
Giải thích được	Có (export_text)	Khó
Feature importance	Tập trung (74% vào 1 feature)	Phân tán đều hơn



Hình 6.7: So sánh các chỉ số đánh giá của Decision Tree và Random Forest

6.7.2 Quyết định chọn mô hình

Mặc dù Random Forest có F1-score cao hơn một chút (0.52 so với 0.49), nhóm chọn **Decision Tree làm mô hình chính** vì các lý do sau:

1. **Hiệu năng thực thi:** Decision Tree suy diễn gần như tức thì (một lần duyệt cây), trong khi Random Forest phải tổng hợp kết quả từ 100 cây. Khi tích hợp vào vòng lặp BFS – nơi mô hình được gọi cho hàng chục hành động mỗi node – tốc độ là yếu tố quan trọng. Thực nghiệm cho thấy BFS+DT mất **385 giây** so với BFS thuần 15 giây, cho thấy chi phí gọi model đã là nút thắt cổ chai; dùng RF sẽ càng chậm hơn.
2. **Hiệu năng tương đương:** Chênh lệch F1 giữa hai mô hình chỉ là 0.03 trên tập test, và gần như bằng nhau trên cross-validation (0.502 vs 0.504 $\pm$ 0.005). Sự khác biệt này không có ý nghĩa thống kê đủ để bù lại chi phí tốc độ.
3. **Khả năng giải thích:** Luật của Decision Tree có thể được trích xuất và đọc được (xem Hình 6.8), giúp kiểm chứng rằng mô hình học được các quy luật có ý nghĩa.

6.7.3 Luật suy diễn của Decision Tree

```
=====
LUẬT DECISION TREE (3 mức đầu)
=====
|--- delta_holes <= 0.50
|   |--- filled_ratio <= 0.22
|   |   |--- valid_actions_after <= 46.50
|   |   |   |--- bbox_after <= 45.00
|   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- bbox_after > 45.00
|   |   |   |   |--- truncated branch of depth 3
|   |   |   |--- valid_actions_after > 46.50
|   |   |   |--- bbox_after <= 20.50
|   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- bbox_after > 20.50
|   |   |   |   |--- truncated branch of depth 3
|   |   |--- filled_ratio > 0.22
|   |   |   |--- valid_actions_before <= 47.50
|   |   |   |   |--- score_gain <= 4.00
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- score_gain > 4.00
|   |   |   |   |   |--- class: 1
|   |   |   |--- valid_actions_before > 47.50
|   |   |   |   |--- lines_cleared <= 0.50
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- lines_cleared > 0.50
|   |   |   |   |   |--- truncated branch of depth 2
|   |--- delta_holes > 0.50
|   |   |--- valid_actions_before <= 46.50
|   |   |   |--- delta_holes <= 1.50
|   |   |   |   |--- y <= 5.50
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- y > 5.50
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |   |--- delta_holes > 1.50
|   |   |   |   |--- class: 0
|   |   |--- valid_actions_before > 46.50
|   |   |   |--- valid_actions_before <= 51.50
|   |   |   |   |--- block_name_Z_270 <= 0.50
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- block_name_Z_270 > 0.50
|   |   |   |   |   |--- truncated branch of depth 2
|   |   |   |--- valid_actions_before > 51.50
|   |   |   |   |--- block_name_L_90 <= 0.50
|   |   |   |   |   |--- truncated branch of depth 3
|   |   |   |   |--- block_name_L_90 > 0.50
|   |   |   |   |   |--- truncated branch of depth 2
```

Hình 6.8: Luật suy diễn của Decision Tree (3 mức đầu)

Một số luật có thể đọc từ cây:

- $\text{delta_holes} > 1.5 \Rightarrow$ lớp 0 (xấu): hành động tạo thêm nhiều hơn 1 vùng cô lập thường là nước xấu.
- $\text{delta_holes} \leq 0.5$ và $\text{filled_ratio} > 0.22$ và $\text{score_gain} > 4 \Rightarrow$ lớp 1 (tốt): không tạo vùng cô lập, bàn cờ đã khá đầy, và có điểm thưởng.
- $\text{valid_actions_before} > 47.5$ và $\text{lines_cleared} > 0.5 \Rightarrow$ lớp 1: hành động xóa được hàng/cột khi còn nhiều lựa chọn là tốt.

6.8 Phân tích kết quả và giới hạn mô hình

6.8.1 Phân tích Precision – Recall

Cả hai mô hình đều có **Recall lớp 1 cao** (≈ 0.85 – 0.88) nhưng **Precision lớp 1 thấp** (≈ 0.34 – 0.37). Điều này có nghĩa:

- Mô hình tìm được phần lớn các hành động thực sự tốt (ít bỏ sót).
- Nhưng trong số các hành động mô hình dự đoán là "tốt", có đến 63% thực ra không tốt (false positive cao).

Trong bối cảnh tích hợp vào BFS, **Recall cao quan trọng hơn Precision**: nếu mô hình bỏ sót hành động tốt (false negative), BFS sẽ không mở rộng nhánh tốt đó. Ngược lại, nếu mô hình đánh giá nhầm một số hành động xấu là tốt (false positive), BFS vẫn sẽ loại bỏ chúng thông qua hàm heuristic.

6.8.2 Nguyên nhân F1-score bị giới hạn ở mức ≈ 0.50

F1 không tăng dù tăng dataset lên 500 states (7,497 mẫu) vì vấn đề nằm ở *chất lượng nhãn*, không phải số lượng. Block Blast là bài toán **sequential decision making**: giá trị thực của một hành động phụ thuộc vào toàn bộ chuỗi nước đi sau đó. Decision Tree chỉ nhìn snapshot tại một thời điểm – không thể capture thông tin về tương lai. Đây là giới hạn cơ bản của kiến trúc phân loại snapshot, không phải lỗi implementation.

$F1 \approx 0.50$ trên bài toán imbalanced (80/20) với label nhiễu từ sequential game là kết quả chấp nhận được và có thể giải thích được

6.9 Tích hợp mô hình vào Best-first Search

Mô hình được tích hợp vào thuật toán tìm kiếm qua hàm `best_first_search_ml()`. Công thức ưu tiên mở rộng node được điều chỉnh như sau:

$$\text{priority}(n) = \underbrace{g(n) + \lambda \cdot |\text{blocks_remaining}|}_{\text{heuristic cơ sở}} - \underbrace{\beta \cdot P(\text{good} \mid \text{action})}_{\text{ML bonus}} \quad (6.1)$$

trong đó:

- $g(n)$ là chi phí đường đi từ gốc đến node n .
- $\lambda = 5$ là trọng số penalize số block còn lại chưa đặt.
- $\beta = 3.0$ (`ml_beta`) là trọng số ảnh hưởng của dự đoán ML.
- $P(\text{good} \mid \text{action}) \in [0, 1]$ là xác suất hành động tốt từ `predict_proba()`.

Node có $P(\text{good})$ cao sẽ có priority thấp hơn, được mở rộng sớm hơn trong hàng đợi ưu tiên. Khi $\beta = 0$, thuật toán trở về BFS thuần; khi β tăng, ML ảnh hưởng mạnh hơn vào thứ tự mở rộng.

6.9.1 Kết quả thực nghiệm so sánh BFS thuần vs BFS + DT

```
... 1. BFS THUẦN (Module A + B + C)
=====
Expanded nodes: 1000
Terminal found: True
Best score: 8
Placed blocks: 4
Remaining blocks: []

Action sequence:
  Step 1: place I_0 at (2, 0)
  Step 2: place T_180 at (2, 4)
  Step 3: place L_90 at (0, 6)
  Step 4: place O at (1, 5)

Final board:
[[0 0 0 0 0 0 1 1]
 [0 0 0 0 0 1 1 1]
 [0 0 0 0 0 1 1 0]
 [0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]]
Thời gian: 15.38s

=====
2. BFS + DECISION TREE (Module A+B+C+D+E)
=====
Expanded nodes: 1000
Terminal found: True
Best score: 8
Placed blocks: 4
Remaining blocks: []

Action sequence:
  Step 1: place I_0 at (2, 4)
  Step 2: place O at (1, 2)
  Step 3: place L_90 at (2, 0)
  Step 4: place T_180 at (2, 1)

Final board:
[[0 0 0 0 0 0 0 0]
 [0 0 1 1 0 0 0 0]
 [0 1 1 1 0 0 0 0]
 [0 1 1 0 0 0 0 0]
 [0 1 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]]
Thời gian: 385.03s

=====
BẢNG TỔNG KẾT SO SÁNH
=====


| Phương pháp | Điểm | Node mở | Terminal |
|-------------|------|---------|----------|
| BFS thuần   | 8    | 1000    | True     |
| BFS + DT    | 8    | 1000    | True     |


```

Hình 6.9: So sánh BFS thuần và BFS + Decision Tree trên bàn cờ demo

Cả hai phương pháp đều tìm được lời giải với điểm số bằng nhau (8 điểm, đặt đủ 4 block). Điều đáng chú ý là **chuỗi hành động khác nhau**: BFS thuần chọn

$I_0 \rightarrow T_{180} \rightarrow L_{90} \rightarrow O$, trong khi BFS+DT chọn $I_0 \rightarrow O \rightarrow L_{90} \rightarrow T_{180}$ – thứ tự đặt block khác nhưng kết quả cuối cùng tương đương. Điều này cho thấy mô hình ML hướng tìm kiếm theo con đường khác nhưng vẫn đạt được mục tiêu.

6.9.2 Kết quả mô phỏng 3 lượt chơi liên tiếp

```
=====
3-TURN GAME FINISHED (model: Random Forest)
=====
Final total score: 40
Total snapshots stored: 15

TURN SUMMARIES

Turn 1
Expanded nodes : 1500
Terminal found : True
Turn score     : 8
Number of moves: 4
  Step 1: place I_0 at (4, 4)
  Step 2: place O at (3, 2)
  Step 3: place L_90 at (4, 0)
  Step 4: place T_180 at (4, 1)

Turn 2
Expanded nodes : 1500
Terminal found : True
Turn score     : 16
Number of moves: 4
  Step 1: place Z_90 at (5, 2)
  Step 2: place J_180 at (2, 2)
  Step 3: place O at (0, 2)
  Step 4: place S_0 at (6, 3)

Turn 3
Expanded nodes : 1500
Terminal found : True
Turn score     : 16
Number of moves: 4
  Step 1: place T_0 at (6, 1)
  Step 2: place I_90 at (0, 1)
  Step 3: place O at (4, 4)
  Step 4: place L_0 at (0, 2)
```

Hình 6.10: Kết quả mô phỏng 3 lượt chơi liên tiếp dùng BFS + ML

Hệ thống đặt thành công toàn bộ block trong cả 3 lượt liên tiếp, đạt tổng 40 điểm. Điều này xác nhận rằng pipeline tích hợp BFS + ML hoạt động ổn định qua nhiều lượt chơi, bàn cờ được cập nhật đúng sau mỗi lượt.

6.10 Các file đầu ra của module học máy

Sau khi huấn luyện, toàn bộ kết quả được lưu vào thư mục `outputs/ml/`:

Bảng 6.8: Các file đầu ra của module học máy

File	Định dạng	Nội dung
blockblast_model.pkl	Pickle (joblib)	Cả hai mô hình DT + RF, model được chọn, danh sách feature
ml_dataset.csv	CSV	Toàn bộ dataset: 7,497 mẫu, đặc trưng thô + nhãn
feature_vectors.npy	NumPy binary	Ma trận đặc trưng đã encode: shape (7497, 47)

6.11 Nhận xét tổng kết

Module học máy trong chương này đã xây dựng thành công một pipeline hoàn chỉnh từ sinh dữ liệu đến tích hợp vào hệ thống tìm kiếm:

1. **Sinh dữ liệu tự động:** 7,497 mẫu được tạo ra mà không cần dataset ngoài, thông qua Teacher Signal (BFS oracle) với ngân sách 1,000 node.
2. **So sánh hai mô hình:** Decision Tree và Random Forest được huấn luyện và đánh giá đầy đủ bằng test split và 5-fold cross-validation.
3. **Lựa chọn có căn cứ:** Decision Tree được chọn làm model chính dựa trên lý do tốc độ suy diễn – yếu tố then chốt khi tích hợp vào BFS.
4. **Tích hợp vào BFS:** ML được nhúng vào hàm ưu tiên của Best-first Search thông qua tham số $\beta = 3.0$, tạo ra thuật toán `best_first_search_ml()`.
5. **Lưu trữ kết quả:** Model, dataset và vector đặc trưng được lưu ra `outputs/ml/` theo đúng yêu cầu đề bài.

Giới hạn và hướng cải thiện: F1-score ≈ 0.50 phản ánh giới hạn cơ bản của phân loại snapshot trên bài toán sequential. Để cải thiện, có thể xem xét: (i) thay Teacher Signal bằng rollout Monte Carlo để nhãn chính xác hơn; (ii) dùng Gradient Boosting thay Decision Tree; hoặc (iii) chuyển sang Reinforcement Learning để học chiến lược toàn cục thay vì phân loại từng hành động độc lập.

Chương 7

Kết luận

7.1 Tóm tắt kết quả đạt được

Đề tài đã xây dựng thành công một hệ thống AI tích hợp năm module để giải bài toán xếp block trên bàn cờ 8×8 theo mô hình Block Blast. Mỗi module đóng một vai trò riêng biệt và bổ sung lẫn nhau trong toàn bộ pipeline ra quyết định:

1. **Biểu diễn & Tìm kiếm (Chương 2):** Bài toán được mô hình hóa rõ ràng qua bộ ba (*State, Action, Goal*). Thuật toán Best-first Search với hàng đợi ưu tiên được triển khai để duyệt không gian trạng thái, tìm chuỗi hành động tối ưu đặt hết block trong một lượt.
2. **Heuristic (Chương 3):** Hàm đánh giá $Eval(n) = g(n) + h(n)$ kết hợp chi phí tích lũy (phạt vùng trống cô lập, diện tích bounding box, thưởng xóa hàng/cột) với ước lượng heuristic theo số block còn lại. Ablation study xác nhận tầm quan trọng của từng thành phần.
3. **Biểu diễn tri thức (Chương 4):** Hệ thống luật IF-THEN được tích hợp trực tiếp vào BFS dưới dạng cơ chế tie-break và cắt nhánh sớm: vị từ Trapped loại bỏ trạng thái có ô bị cô lập không thể lấp, vị từ Completes ưu tiên hành động hoàn tất hàng/cột.
4. **Mạng Bayes (Chương 5):** Mô hình Bayes năm biến (*Density, Fragmentation, LineClear, ValidMoveLevel, StuckRisk*) được xây dựng từ 50.000 mẫu rollout mô phỏng. Bảng CPD học bằng đếm tần suất cho phép suy diễn $P(\text{StuckRisk} \mid \text{board state})$ để đánh giá rủi ro bị kẹt.
5. **Học máy (Chương 6):** Decision Tree Classifier được huấn luyện trên 7.497 mẫu (*state, action*) có nhãn từ Teacher Signal (BFS oracle). Mô hình dự đoán $P(\text{good} \mid \text{action})$ và được tích hợp vào hàm ưu tiên BFS qua tham số $\beta = 3.0$, tạo ra biến thể `best_first_search_ml()`.

7.2 Kết quả thực nghiệm

Hệ thống hoàn chỉnh đặt thành công toàn bộ block trong mọi lượt thử nghiệm. Mô phỏng 3 lượt liên tiếp với BFS + Decision Tree đạt tổng 40 điểm, terminal found ở cả ba lượt. So sánh BFS thuần và BFS + ML trên cùng trạng thái cho thấy cả hai đạt điểm bằng nhau (8 điểm), xác nhận ML hướng tìm kiếm theo con đường khác nhưng không làm giảm chất lượng lời giải.

Decision Tree đạt Accuracy 0.641 và F1-score 0.487 trên tập test (5-fold CV: $F1 = 0.502 \pm 0.004$), được chọn làm model chính do tốc độ suy diễn vượt trội so với Random Forest – yếu tố then chốt khi tích hợp vào vòng lặp BFS.

7.3 Hạn chế

- **F1-score bị giới hạn ở ≈ 0.50 :** Do Block Blast là bài toán sequential decision making, Decision Tree chỉ nhìn snapshot tại một thời điểm nên không thể capture đầy đủ thông tin về chuỗi hành động tương lai.
- **Teacher Signal mang circular bias:** Nhận được sinh từ BFS oracle dùng cùng heuristic với hệ thống tìm kiếm, khiến mô hình có xu hướng học lại heuristic thay vì học chiến lược độc lập.
- **Mạng Bayes và ML chưa kết nối:** Hai module hoạt động song song và độc lập; tích hợp $P(\text{StuckRisk})$ từ Bayes vào priority function của BFS có thể cải thiện thêm chất lượng tìm kiếm.
- **Chi phí thực thi:** Việc gọi mô hình ML cho mỗi hành động trong vòng lặp BFS làm tăng thời gian chạy đáng kể (từ 15 giây lên 385 giây trong thử nghiệm demo).

7.4 Hướng phát triển

- Thay thế phân loại snapshot bằng **Reinforcement Learning** (Q-learning, Policy Gradient) để học chiến lược toàn cục qua nhiều lượt chơi.
- Kết hợp $P(\text{StuckRisk})$ từ Mạng Bayes vào hàm ưu tiên BFS để hai module D và E hỗ trợ nhau thực sự.
- Áp dụng **Monte Carlo Tree Search** thay BFS để cân bằng tốt hơn giữa khai phá và khai thác.
- Mở rộng tập đặc trưng bằng các đặc trưng hướng tương lai (look-ahead features) để giảm circular bias trong Teacher Signal.

Phụ lục A

Phân chia công việc nhóm

Thứ tự	Họ và tên	Mã số sinh viên	Công việc	Đóng góp
1	Phạm Ngọc Long	2211894	Mô hình hóa bài toán, Heuristic, Logic	100%
2	Nguyễn Ngọc Duy	2210522	Heuristic, Logic, Bayes	100%
3	Võ Quốc Phong	2352913	Heuristic - Ablation, Teacher Signal - Machine Learning Study	100%
4	Nguyễn Tấn Đạt	2352234	Mô hình hóa bài toán, Machine Learning	100%

Bảng A.1: Bảng phân công công việc và tỷ lệ đóng góp

Phụ lục B

Lịch sử sửa đổi

Họ và Tên	Ngày	Lý do thay đổi	Phiên bản
Nguyen Ngoc Duy	01/03/2026	Khởi tạo document	0.1
Pham Ngoc Long	xx/03/2026	Triển khai,	0.1.1

Phụ lục C

Lịch sử cuộc họp

Người tham gia	Ngày	Nội dung cuộc họp	Phiên bản
4 thành viên	01/03/2026	Khởi tạo document	0.1
4 thành viên	xx/03/2026	Triển khai,	0.1.1
Nguyen Ngoc Duy, Phạm Ngọc Long, Võ Quốc Phong	07/04/2026	Confirm xong lý thuyết chương 3, chương 4 có 1 phần hoàn thiện và có thể bổ sung thêm. Chương 6 đã bắt đầu tìm hiểu và cần code thêm	0.1